

RucBase安装以及使用(Docker+VsCode环境)

一、安装并更新

1.1、命令行

二、获取github上最新的源文件、编译

2.1、vscode环境

2.2、git获取最新源码

2.3、重新编译测试

三、启动

3.1、服务端

3.2、客户端

附录

一、安装并更新

1.1、命令行

▼ 拉取镜像

Shell |

```
1 docker pull djb2023/rucbase_lab:djb-ssh
```

▼ 创建容器

Shell |

```
1 docker run -dit --name=rucbase djb2023/rucbase_lab:djb-ssh
```

二、获取github上最新的源文件、编译

2.1、vscode环境

- 下载docker插件
- 进入/rucbase-lab目录

2.2、git获取最新源码

git拉取

Shell

```
1 git pull
```

PROBLEMS

OUTPUT

DEBUG CONSOLE

TERMINAL

PORTS

```
● root@6060a73d16eb:/rucbase-lab# git pull
remote: Enumerating objects: 363, done.
remote: Counting objects: 100% (285/285), done.
remote: Compressing objects: 100% (115/115), done.
remote: Total 363 (delta 184), reused 263 (delta 170), pack-reused 78
Receiving objects: 100% (363/363), 734.36 KiB | 3.71 MiB/s, done.
Resolving deltas: 100% (198/198), completed with 42 local objects.
From https://github.com/ruc-deke/rucbase-lab
   9b52c0d..2921218  main       -> origin/main
Updating 9b52c0d..2921218
Fast-forward
 .gitignore
 CMakeLists.txt
```

2.3、重新编译测试

进入/rucbase-lab/build目录

Shell

```
1 cd build
```

删除旧编译文件

Shell

```
1 rm -rf ./*
```

▼

Shell

```
1 cmake ..
```

▼

Shell

```
1 make
```

三、启动

3.1、服务端

▼ 启动服务端	Shell
1 <code>./rmdb test</code>	

3.2、客户端

▼ 进入客户端目录	Shell
1 <code>cd /rucbase-lab/rucbase_client/</code>	

▼ 创建build	Shell
1 <code>mkdir build</code>	
2 <code>cd build</code>	

▼ 编译	Shell
1 <code>cmake ..</code>	
2 <code>make</code>	

▼ 启动客户端	Shell
1 <code>./rucbase_client test</code>	

附录

在vscode中用于调试的launch.json文件

```
1 {
2   "version": "0.2.0",
3   "configurations": [
4     {
5       "name": "rucbase",
6       "type": "cppdbg", // 调试类型
7       "request": "launch",
8       "program": "${workspaceFolder}/build/bin/disk_manager_test", // 要调
        试的可执行文件的路径
9       "args": [
10        "testdb"
11      ], // 传递给程序的命令行参数
12      "stopAtEntry": false, // 是否在程序入口处停止
13      "cwd": "${workspaceFolder}/build/", // 程序运行时的工作目录
14      "environment": [],
15      "externalConsole": false,
16      "MIMode": "gdb",
17      "setupCommands": [
18        {
19          "description": "Enable pretty-printing for gdb",
20          "text": "-enable-pretty-printing",
21          "ignoreFailures": true // 是否忽略命令失败
22        }
23      ],
24      "preLaunchTask": "",
25      "targetArchitecture": "x86_64",
26      "miDebuggerPath": "/usr/bin/gdb" // gdb 调试器的路径
27    }
28  ]
29 }
30
```

视频

https://www.bilibili.com/video/BV1Nc41147Pd/?spm_id_from=333.337.search-card.all.click&vd_source=0c3da11c093b383850aaa5eabed3d89d

c_cpp_properties.json

```
1 {
2   "configurations": [
3     {
4       "name": "Linux",
5       "includePath": [
6         "${workspaceFolder}/**",
7         "/usr/include",
8         "/usr/local/include",
9         "/usr/include/x86_64-linux-gnu",
10        "/usr/lib/gcc/x86_64-linux-gnu/13/include"
11      ],
12      "defines": [],
13      "compilerPath": "/usr/bin/gcc",
14      "cStandard": "c17",
15      "cppStandard": "c++17",
16      "intelliSenseMode": "gcc-x64"
17    }
18  ],
19  "version": 4
20 }
21
```